

BASH SCRIPTING FUNDAMENTALS

Shell Basics

Terminal navigation, commands, PATH, and your first scripts

Merit Advisory
Bash Scripting Fundamentals Bootcamp
Beginner-friendly lecture materials
July 2026

July 2026



Agenda

- 1 Section 1: Shell and Terminal Foundations — 7 topics
- 2 Section 2: Navigation and Paths — 7 topics
- 3 Section 3: Listing and Inspecting Directories — 7 topics
- 4 Section 4: Creating and Moving Files — 7 topics
- 5 Section 5: Removing and Viewing Text — 7 topics
- 6 Section 6: Finding Commands and Getting Help — 7 topics
- 7 Section 7: Output and First Scripts — 7 topics
- 8 Section 8: Environment and Identity — 7 topics

- 9 Section 9: System Awareness and Patterns — 7 topics
- 1 Section 10: Safe Command Habits — 7 topics
- 0
- 1 Section 11: Keyboard Workflow — 7 topics
- 1
- 1 Section 12: Practice Lab and Checkpoints — 7 topics
- 2

1 Section 1

Shell and Terminal Foundations

1.1 What Is a Shell

- A shell is a program that accepts commands and asks the operating system to run them.

Print a message

```
echo "Hello from the shell"
```

- It gives you a text-based way to open files, start programs, and inspect the computer.

Run a simple command

```
date
```

1.2 What Is Bash

- Bash is a common shell used on Linux, macOS, and many server environments.

Show Bash path

```
command -v bash
```

- It can run commands interactively and also read commands from script files.

Show Bash version

```
bash --version
```

1.3 Terminal Versus Shell

- A terminal is the window or app where you type text commands.

Show terminal type

```
echo "$TERM"
```

- The shell is the program running inside that terminal and interpreting your commands.

Show current shell

```
echo "$SHELL"
```

1.4 Prompt - Where Commands Begin

- The prompt is the text shown before your cursor when the shell is ready.

Type after the prompt

```
whoami
```

- Prompts often include your username, host, current folder, or a symbol such as dollar sign.

Show prompt variable

```
echo "$PS1"
```


1.5 Interactive Shell Versus Script

- An interactive shell runs one command at a time as you type it.

Run interactively

```
echo "Testing one command"
```

- A script is a file of commands that Bash runs from top to bottom.

Run a script file

```
bash hello.sh
```

1.6 Command Shape

- Most shell commands start with a command name followed by options and arguments.

Command with option

```
ls -l
```

- Options usually change behavior, while arguments usually name files, folders, or values.

Command with argument

```
mkdir practice
```

1.7 Tab Completion

- Tab completion asks the shell to finish a command name, file name, or folder name.

Complete a directory

```
cd Doc<Tab>
```

- It saves typing and helps avoid spelling mistakes in long paths.

Complete a command

```
hos<Tab>
```

2 Section 2

Navigation and Paths

2.1 pwd - Print Working Directory

- The working directory is the folder your shell is currently using.

Show current path

```
pwd
```

- The pwd command prints that folder as a full path.

Print path in a message

```
echo "You are in: $(pwd)"
```

2.2 Absolute Paths

- An absolute path starts at the root directory and names every folder needed to reach a file.

List an absolute path

```
ls /tmp
```

- On Linux, absolute paths begin with a slash.

Change to home by path

```
cd "$HOME"
```

2.3 Relative Paths

- A relative path starts from your current working directory.

List a child folder

```
ls scripts
```

- Relative paths are shorter when you are already near the files you want.

Open a nearby file

```
cat notes.txt
```

2.4 cd - Go Home

- The cd command changes your current working directory.

Return home

```
cd
```

- Running cd with no argument returns you to your home directory.

Use tilde home

```
cd ~
```


2.5 cd .. - Go to Parent

- Two dots mean the parent directory of the current directory.

Go up one level

```
cd ..
```

- Using cd .. moves one level upward in the folder tree.

Go up two levels

```
cd ../../
```

2.6 cd - - Return to Previous Directory

- A single dash after cd means the previous working directory.

Switch back

```
cd -
```

- This is useful when you need to jump between two folders repeatedly.

Visit and return

```
cd /tmp  
cd -
```

2.7 Working Directory in Scripts

- A script usually starts in the directory where you launched it, not where the script file lives.

Show script location effect

```
printf "cwd=%s\n" "$(pwd)"
```

- Commands inside the script use that starting directory unless the script changes it.

Run from another folder

```
cd /tmp  
bash /path/to/script.sh
```

3 Section 3

Listing and Inspecting Directories

3.1 ls - List Directory Names

- The ls command lists files and folders in a directory.

List current directory

```
ls
```

- Without arguments, ls lists the current working directory.

List another directory

```
ls /tmp
```

3.2 ls -l - Long Listing

- The -l option shows one item per line with extra details.

[Show details](#)

```
ls -l
```

- Long listings include permissions, owner, size, and modification time.

[Show details for one path](#)

```
ls -l notes.txt
```

3.3 ls -a - Hidden Files

- Hidden file names begin with a dot on Unix-like systems.

Include hidden files

```
ls -a
```

- The -a option includes hidden files in the listing.

Long hidden listing

```
ls -la
```

3.4 ls -h - Human Sizes

- The -h option makes file sizes easier for people to read.

Long human listing

```
ls -lh
```

- It is most useful together with -l because long listings display sizes.

All details with sizes

```
ls -lah
```


3.5 tree - Directory Overview

- The tree command shows folders and files as a visual hierarchy.

Show current tree

```
tree
```

- It helps beginners understand how a project is arranged.

Limit tree depth

```
tree -L 2
```

3.6 File Type Characters

- The first character in `ls -l` output describes the kind of item.

[See type characters](#)

```
ls -l
```

- A dash usually means a regular file, `d` means directory, and `l` means symbolic link.

[Show directory entry](#)

```
ls -ld .
```

3.7 Permissions Overview

- Permissions describe who can read, write, or execute a file.

[View permissions](#)

```
ls -l script.sh
```

- The letters r, w, and x stand for read, write, and execute.

[View directory permissions](#)

```
ls -ld .
```

4 Section 4

Creating and Moving Files

4.1 mkdir - Create a Directory

- The mkdir command creates a new directory.

Create one directory

```
mkdir lab
```

- It is commonly used to prepare a workspace before creating files.

Create two directories

```
mkdir input output
```

4.2 mkdir -p - Create Parent Directories

- The -p option creates missing parent directories along the path.

Create nested folders

```
mkdir -p lab/raw
```

- It also avoids an error when the target directory already exists.

Create repeatable output

```
mkdir -p reports/daily
```

4.3 touch - Create an Empty File

- The touch command creates an empty file when the name does not exist.

Create one file

```
touch notes.txt
```

- If the file already exists, touch updates its modification time.

Create several files

```
touch one.txt two.txt
```

4.4 cp - Copy a File

- The cp command copies a file to a new name or location.

Copy with new name

```
cp notes.txt notes.bak
```

- Copying keeps the original file in place.

Copy into directory

```
cp notes.txt lab/
```


4.5 cp -r - Copy a Directory

- Directories contain other entries, so cp needs recursive mode to copy them.

Copy a directory

```
cp -r lab lab-backup
```

- The -r option copies a directory and its contents.

Copy into backups

```
cp -r reports backups/
```

4.6 mv - Rename a File

- The mv command can rename a file in the same directory.

Rename a file

```
mv draft.txt final.txt
```

- Renaming changes the name but does not copy the file contents.

Rename a directory

```
mv lab practice-lab
```

4.7 mv - Move a File

- The mv command also moves files or directories to another location.

Move file into folder

```
mv final.txt reports/
```

- Moving changes where the item lives in the directory tree.

Move directory into folder

```
mv practice-lab archive/
```

5 Section 5

Removing and Viewing Text

5.1 rm - Remove a File

- The rm command deletes files.

Remove one file

```
rm old.txt
```

- Deleted files are not normally moved to a desktop trash folder.

Preview then remove

```
ls old.txt  
rm old.txt
```

5.2 rm -r - Remove a Directory Tree

- The -r option removes a directory and everything inside it.

Remove a practice folder

```
rm -r lab
```

- Recursive removal is powerful and can delete many files quickly.

Remove generated reports

```
rm -r reports/tmp
```

5.3 cat - Print a File

- The cat command prints file contents directly to the terminal.

Show a text file

```
cat notes.txt
```

- It is best for short files that fit comfortably on one screen.

Combine two files

```
cat header.txt body.txt
```

5.4 less - Page Through Text

- The less command opens text in a scrollable pager.

Open a long file

```
less system.log
```

- You can move with arrow keys, Page Up, and Page Down.

Read command output

```
ls -la | less
```


5.5 more - Simple Pager

- The more command is an older pager that displays text page by page.

Page through a file

```
more README.txt
```

- It is less flexible than less but still appears on many systems.

Page through listing

```
ls -la | more
```

5.6 head - First Lines

- The head command prints the beginning of a file.

Show first lines

```
head app.log
```

- By default, it shows the first ten lines.

Show first five lines

```
head -n 5 app.log
```

5.7 tail - Last Lines

- The tail command prints the end of a file.

Show last lines

```
tail app.log
```

- It is often used to inspect the newest entries in a log file.

Show last twenty lines

```
tail -n 20 app.log
```

6 Section 6

Finding Commands and Getting Help

6.1 which - Find a Command Path

- The which command searches PATH and prints the executable it would run.

Find Bash

```
which bash
```

- It is useful when you want to know where a command comes from.

Find Python

```
which python3
```

6.2 type - Explain a Command

- The type command tells Bash how it understands a command name.

Explain cd

```
type cd
```

- It can identify builtins, aliases, functions, and executable files.

Explain ls

```
type ls
```

6.3 command -v - Portable Command Check

- The command -v form asks the shell to locate a command in a script-friendly way.

Check for sed

```
command -v sed
```

- It works well when a script needs to check whether a tool is available.

Use in a condition

```
command -v git >/dev/null
```

6.4 man - Manual Pages

- The man command opens the system manual for many commands.

[Open ls manual](#)

```
man ls
```

- Manual pages describe options, arguments, behavior, and related commands.

[Open Bash manual](#)

```
man bash
```


6.5 --help - Quick Command Help

- Many commands accept --help to print a short usage summary.

[Show mkdir help](#)

```
mkdir --help
```

- Help output is usually faster to scan than a full manual page.

[Show cp help](#)

```
cp --help
```

6.6 history - Previous Commands

- The history command lists commands you typed earlier in the shell.

Show recent history

```
history
```

- History helps you repeat, review, and learn from previous work.

Show last five entries

```
history 5
```

6.7 clear - Clean the Screen

- The clear command redraws the terminal with an empty view.

Clear terminal

```
clear
```

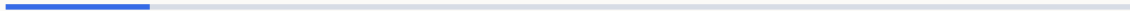
- It does not delete files or erase command history.

Print after clear

```
clear  
echo "Ready"
```

7 Section 7

Output and First Scripts



7.1 echo - Print Text

- The echo command prints text followed by a newline.

Print a greeting

```
echo "Hello"
```

- It is useful for quick messages and simple checks.

Print two words

```
echo "Bash basics"
```

7.2 printf - Formatted Output

- The printf command prints text using a format string.

Print with newline

```
printf "Hello\n"
```

- It does not add a newline unless the format asks for one.

Print a value

```
printf "User: %s\n" "$USER"
```

7.3 Comments in Bash

- A comment explains code for humans and is ignored by Bash.

Single comment

```
# Create the output directory  
mkdir -p output
```

- Comments begin with a hash character when it is not inside quotes.

Comment after command

```
date # Show the current date
```

7.4 exit - End a Shell or Script

- The exit command ends the current shell or script.

Exit successfully

```
exit 0
```

- A numeric exit status can tell other programs whether the script succeeded.

Exit with failure

```
exit 1
```


7.5 Shebang for Bash Scripts

- A shebang is the first line that tells the system which interpreter should run a script.

Bash shebang

```
#!/usr/bin/env bash
```

- The form using env finds Bash through the current PATH.

First script lines

```
#!/usr/bin/env bash  
echo "Hello"
```

7.6 chmod +x - Make a Script Executable

- The chmod command changes file permissions.

Make executable

```
chmod +x hello.sh
```

- Adding +x gives permission to execute a script file.

Check permission

```
ls -l hello.sh
```

7.7 bash Script Versus ./script

- Running `bash script.sh` asks Bash to read the file even if it is not executable.

Run with Bash

```
bash hello.sh
```

- Running `./script.sh` asks the system to execute the file directly.

Run directly

```
./hello.sh
```

8 Section 8

Environment and Identity

8.1 PATH Explained

- PATH is a list of directories the shell searches for command names.

Print PATH

```
echo "$PATH"
```

- When you type ls, Bash looks through PATH to find the program or builtin behavior.

Split PATH lines

```
printf "%s\n" "$PATH"
```

8.2 PATH Search Order

- Bash checks PATH directories from left to right.

[Show command chosen](#)

```
command -v python3
```

- If two directories contain the same command name, the earlier directory wins.

[Show PATH value](#)

```
echo "$PATH"
```

8.3 env - Show Environment

- The env command prints environment variables passed to programs.

Print environment

```
env
```

- Environment variables are name and value pairs available to child processes.

Page environment

```
env | less
```

8.4 printenv - Print Environment Values

- The printenv command prints all environment variables or one selected variable.

Print HOME

```
printenv HOME
```

- It is a clear way to check values such as HOME, USER, and PATH.

Print all variables

```
printenv
```


8.5 export - Share a Variable

- The export command marks a shell variable to be passed to child programs.

Export one value

```
export APP_ENV=dev
```

- It is often used when a script or tool reads configuration from the environment.

Use exported value

```
bash -c 'echo "$APP_ENV"'
```

8.6 whoami - Current User

- The whoami command prints the username of the current effective user.

Show current user

```
whoami
```

- It helps you confirm whether you are working as yourself or another account.

Use in a message

```
echo "Running as $(whoami)"
```

8.7 hostname - Current Machine Name

- The hostname command prints the name of the computer or container.

Show host name

```
hostname
```

- It helps when you work across multiple servers or remote sessions.

Print with user

```
echo "$(whoami)@$(hostname)"
```

9 Section 9

System Awareness and Patterns

9.1 date - Current Date and Time

- The date command prints the current date and time.

Show date

```
date
```

- It can also format timestamps for filenames and logs.

Create compact date

```
date +%Y%m%d"
```

9.2 df - Disk Free Space

- The df command reports free and used space on mounted filesystems.

Show disk space

```
df -h
```

- The -h option makes the sizes easier to read.

Check current filesystem

```
df -h .
```

9.3 du - Directory Usage

- The du command estimates how much disk space files and directories use.

Summarize current folder

```
du -sh .
```

- The -s option summarizes instead of listing every child path.

Summarize directories

```
du -sh */
```

9.4 find - Search by Path Rules

- The find command walks directories and matches files by rules you choose.

Find text files

```
find . -name "*.txt"
```

- It can search by name, type, size, time, and many other properties.

Find directories

```
find . -type d
```


9.5 locate Versus find

- The locate command searches a prebuilt database of file paths.

Use locate

```
locate bashrc
```

- Locate can be faster than find, but its database may be out of date.

Use live search

```
find . -name ".bashrc"
```

9.6 Wildcard Star

- The star wildcard matches zero or more characters in a file name.

List text files

```
ls *.txt
```

- The shell expands the wildcard before the command receives the arguments.

Remove backup files

```
rm *.bak
```

9.7 Wildcard Question and Brackets

- The question mark wildcard matches exactly one character.

Match one digit

```
ls file?.txt
```

- Bracket patterns match one character from a selected set or range.

Match a range

```
ls report-[1-3].txt
```

10

Section 10

Safe Command Habits

10.1 Paths with Spaces

- Spaces separate words in shell commands.

Create spaced file

```
touch "my notes.txt"
```

- A file name with spaces must be quoted so the shell treats it as one path.

List spaced file

```
ls -l "my notes.txt"
```

10.2 Case-Sensitive Names

- Linux file names are usually case-sensitive.

Create two names

```
touch Notes.txt notes.txt
```

- Notes.txt and notes.txt can be two different files.

List both names

```
ls *otes.txt
```

10.3 Preview Before Destructive Commands

- Destructive commands change or remove data.

Preview matches

```
ls *.tmp
```

- Previewing paths with `ls` or `find` helps confirm your target before you delete.

Delete after preview

```
rm *.tmp
```

10.4 -- - End of Options

- A double dash tells many commands that options are finished.

Remove dashed name

```
rm -- -draft.txt
```

- It helps when a file name begins with a dash and could look like an option.

List dashed name

```
ls -- -draft.txt
```


10.5 Command Not Found

- Command not found means Bash could not locate what you typed as a command.

Check command

```
command -v tree
```

- The cause may be a spelling mistake, a missing program, or a PATH issue.

Inspect PATH

```
echo "$PATH"
```

10.6 Permission Denied

- Permission denied means the system refused the requested access.

Inspect a script

```
ls -l run.sh
```

- For scripts, the common cause is missing execute permission.

Add execute permission

```
chmod +x run.sh
```

10.7 Read Error Messages

- Error messages often name the command, path, and kind of problem.

Trigger missing file

```
cat missing.txt
```

- Reading the first error carefully can save a lot of guessing.

Check current path

```
pwd
```

11

Section 11

Keyboard Workflow

11.1 Ctrl-C - Interrupt a Command

- Ctrl-C sends an interrupt signal to the running foreground command.

Start a wait

```
sleep 30
```

- It is the usual way to stop a command that is taking too long.

Interrupt it

```
Press Ctrl-C
```

11.2 Ctrl-D - End Input

- Ctrl-D tells the terminal there is no more input to read.

Exit an empty prompt

```
Press Ctrl-D
```

- At an empty shell prompt, it usually exits the shell.

End cat input

```
cat  
Press Ctrl-D
```

11.3 Ctrl-L - Clear the View

- Ctrl-L clears the terminal view like the clear command.

Clear by shortcut

Press Ctrl-L

- It keeps the current command line available while cleaning the screen.

Clear by command

```
clear
```

11.4 History Navigation Keys

- The up and down arrow keys move through recent commands.

Recall previous command

```
Press Up
```

- Reusing history saves typing and reduces spelling mistakes.

Run history list

```
history 10
```


11.5 Editing the Command Line

- Basic command-line editing lets you fix a command before running it.

Move within command

Press Left or Right

- Left and right arrows move the cursor within the current command.

Fix then run

```
echo "helo"  
# edit to hello
```

11.6 Aliases Intro

- An alias is a short name that expands to another command.

Create an alias

```
alias ll='ls -la'
```

- Aliases are convenient for interactive shortcuts.

Use the alias

```
ll
```

11.7 Removing an Alias

- The `unalias` command removes an alias from the current shell session.

Remove one alias

```
unalias ll
```

- Removing an alias helps when a shortcut hides the real command behavior.

List aliases

```
alias
```

1 2

Section 12

Practice Lab and Checkpoints

12.1 Create a Practice Lab

- A practice lab is a safe folder where you can try commands freely.

Create lab folders

```
mkdir -p bash-lab/input bash-lab/output
```

- Keeping practice work separate lowers the risk of changing important files.

Enter the lab

```
cd bash-lab
```

12.2 Make Practice Files

- Small practice files let you test copy, move, view, and remove commands.

Create sample files

```
touch input/a.txt input/b.txt
```

- Using predictable names makes wildcard behavior easier to see.

Add sample text

```
printf "one\ntwo\n" > input/a.txt
```

12.3 Hidden Configuration Files

- Many shell settings live in hidden files in your home directory.

List hidden home files

```
ls -la ~
```

- Examples include files such as .bashrc and .profile.

Check bashrc

```
ls -l ~/.bashrc
```

12.4 File Naming Habits

- Simple names make shell commands easier to read and type.

Create simple names

```
touch report-01.txt
```

- Lowercase letters, numbers, dashes, and underscores are usually beginner-friendly.

Create underscore name

```
mkdir daily_reports
```


12.5 Checkpoint - Navigation

- A navigation checkpoint confirms that you can move around safely.

Run navigation check

```
pwd  
cd ..  
pwd
```

- You should be able to print your location, move to a parent folder, and return home.

Return home and check

```
cd  
pwd
```

12.6 Checkpoint - File Operations

- A file operation checkpoint confirms that you can create, copy, move, and remove items.

Create and copy

```
touch a.txt  
cp a.txt b.txt
```

- Use only practice files until each command feels predictable.

Move and remove

```
mv b.txt output.txt  
rm output.txt
```

12.7 Checkpoint - Help and Discovery

- A help checkpoint confirms that you can discover commands without memorizing everything.

Identify a command

```
type printf
```

- Use type or command -v to identify commands before reading deeper help.

Open quick help

```
printf --help
```

NEXT STEP

Practice every example in your terminal

Then open the next module deck

Merit Advisory

01 · Shell Basics

